Optimisation SlowSite — audit, problèmes, corrections

Rapport avant / après — R3.14 Hébergement, BUT MMI Béziers

- Dépôt (fork) : https://github.com/sup3rpanini/R314-optimisation-site
- Site publié : https://sup3rpanini.github.io/R314-optimisation-site/
- Corrections : branche `optimisations`, un commit par optimisation.

1. Audit initial

Conditions de mesure

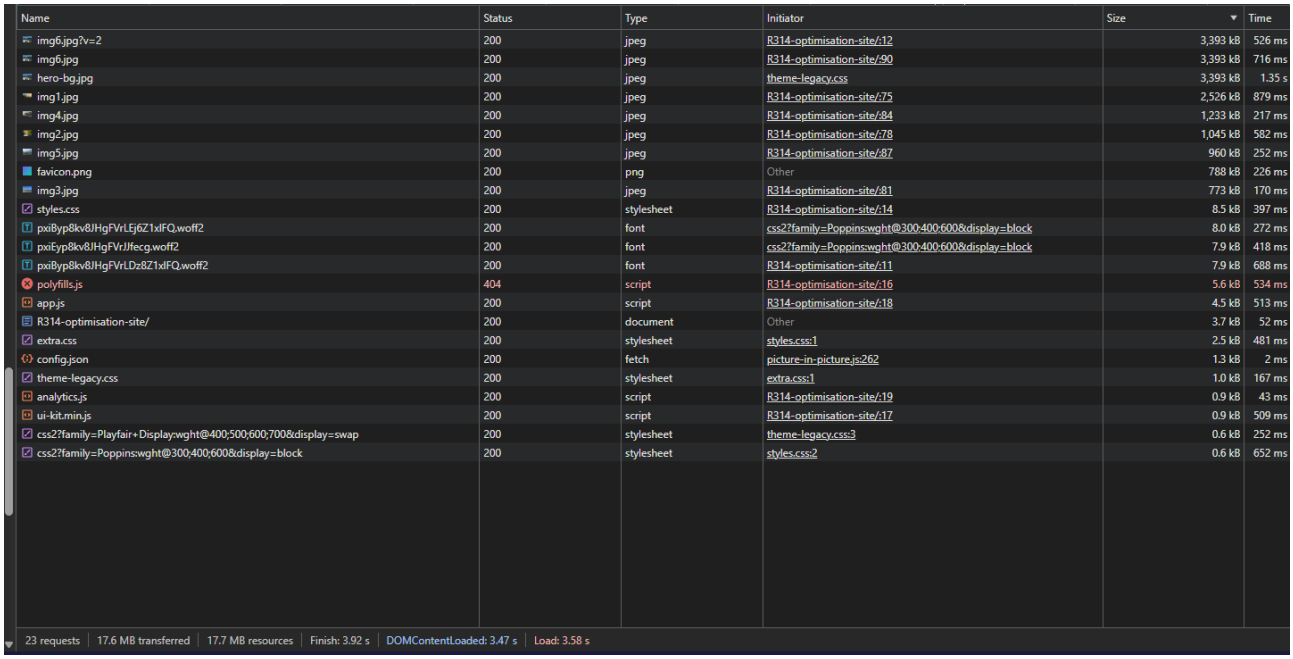
Source	Conditions
DevTools (Réseau, Performances, Lighthouse)	Site déployé, navigation privée, cache désactivé. Lighthouse 13.4.1 : Moto G Power + 4G lente (mobile), 1350×940 (ordinateur). Performances : CPU ×4, avec défilement
PageSpeed Insights	Site déployé, 2 passages : Lighthouse 13.5.0 puis 13.6.0 (1er novembre 2026, 21 h 56)
Exécution JS longue	Chromium, serveur local `python -m http.server`, fenêtre 1350×940 (> 1024 px, voir P19), 30 s avec défilement, sans ralentissement CPU

Limites.

- Pas de données de terrain (CrUX) : le site n'a pas assez de trafic, donc tout l'audit repose sur des données de laboratoire.
- Les temps varient fortement d'un passage à l'autre (TBT desktop : 30, 360 puis 510 ms ; LCP mobile : 12,1 puis 66,8 s). On raisonne en ordres de grandeur. Le poids et le nombre de requêtes, eux, sont stables au kilo-octet près.
- L'enregistrement « recharger et profiler » des DevTools s'arrête de lui-même après le chargement (~9 s). Le minuteur d'`analytics.js`, qui démarre plus tard, est donc documenté par la mesure longue en local et par PSI (voir P19).

Toutes les captures sont reproduites dans ce rapport ; les originaux, y compris les catégories hors périmètre (accessibilité, SEO), sont dans le dossier `audit/` de l'archive.

Réseau (avant) : DevTools, site déployé



Name	Status	Type	Initiator	Size	Time
img6.jpg?v=2	200	jpeg	R314-optimisation-site/:12	3,393 kB	526 ms
img6.jpg	200	jpeg	R314-optimisation-site/:90	3,393 kB	716 ms
hero-bg.jpg	200	jpeg	theme-legacy.css	3,393 kB	1.35 s
img1.jpg	200	jpeg	R314-optimisation-site/:75	2,526 kB	879 ms
img4.jpg	200	jpeg	R314-optimisation-site/:84	1,233 kB	217 ms
img2.jpg	200	jpeg	R314-optimisation-site/:78	1,045 kB	582 ms
img5.jpg	200	jpeg	R314-optimisation-site/:87	960 kB	252 ms
favicon.png	200	png	Other	788 kB	226 ms
img3.jpg	200	jpeg	R314-optimisation-site/:81	773 kB	170 ms
styles.css	200	stylesheet	R314-optimisation-site/:14	8.5 kB	397 ms
pxiByp8kv8JHgFVrLEj6Z1xlFQ.woff2	200	font	css2?family=Poppins:wght@300;400;600&display=block	8.0 kB	272 ms
pxiEyp8kv8JHgFVrJJfecg.woff2	200	font	css2?family=Poppins:wght@300;400;600&display=block	7.9 kB	418 ms
pxiByp8kv8JHgFVrLDz8Z1xlFQ.woff2	200	font	R314-optimisation-site/:11	7.9 kB	688 ms
polyfills.js	404	script	R314-optimisation-site/:16	5.6 kB	534 ms
app.js	200	script	R314-optimisation-site/:18	4.5 kB	513 ms
R314-optimisation-site/	200	document	Other	3.7 kB	52 ms
extra.css	200	stylesheet	styles.css:1	2.5 kB	481 ms
config.json	200	fetch	picture-in-picture.js:262	1.3 kB	2 ms
theme-legacy.css	200	stylesheet	extra.css:1	1.0 kB	167 ms
analytics.js	200	script	R314-optimisation-site/:19	0.9 kB	43 ms
ui-kit.min.js	200	script	R314-optimisation-site/:17	0.9 kB	509 ms
css2?family=Playfair+Display:wght@400;500;600;700&display=swap	200	stylesheet	theme-legacy.css:3	0.6 kB	252 ms
css2?family=Poppins:wght@300;400;600&display=block	200	stylesheet	styles.css:2	0.6 kB	652 ms

23 requests | 17.6 MB transferred | 17.7 MB resources | Finish: 3.92 s | DOMContentLoaded: 3.47 s | Load: 3.58 s

Figure 1 : Onglet Réseau, avant : tri par taille décroissante, barre de total en bas

23 requêtes, 17,6 Mo transférés (17,7 Mo de ressources). DOMContentLoaded à 3,47 s, load à 3,58 s. Les plus lourdes :

Ressource	Taille	Remarque
img6.jpg?v=2	3 393 kB	**doublon** de img6.jpg (preload)
img6.jpg	3 393 kB	
hero-bg.jpg	3 393 kB	**le même fichier**, affiché à 4 % d'opacité
img1.jpg	2 526 kB	3968 px pour un affichage d'environ 388 px
img4.jpg	1 233 kB	
img2.jpg	1 045 kB	
img5.jpg	960 kB	
favicon.png	788 kB	512×512 pour un affichage en 16 px
img3.jpg	773 kB	
polyfills.js	**404**	bloquant dans le `<head>`

La colonne Initiator montre la chaîne critique : `styles.css:1` → `extra.css` → `extra.css:1` → `theme-legacy.css` → `theme-legacy.css:3` → Google Fonts (Playfair), et `styles.css:2` → Google Fonts (Poppins) → woff2.

Lighthouse et PageSpeed Insights (avant)

Outil	Profil	Score	FCP	LCP	TBT	CLS	Speed Index
Lighthouse DevTools	Mobile	**31**	3,5 s	**68,6 s**	470 ms	0,304	23,4 s
Lighthouse DevTools	Ordinateur	**58**	1,3 s	12,7 s	30 ms	0,118	6,7 s
PSI, passage 1	Mobile	63	2,4 s	12,1 s	160 ms	0	7,3 s
PSI, passage 1	Ordinateur	61	1,0 s	3,1 s	360 ms	0	2,1 s
PSI, passage 2	Mobile	**60**	1,7 s	**66,8 s**	220 ms	0,011	17,5 s
PSI, passage 2	Ordinateur	**43**	1,0 s	12,2 s	**510 ms**	0	4,2 s

Rapports PSI du passage 2 : mobile, ordinateur.

Ce que disent les rapports :

- **Élément LCP :** `<div class="hero">` dans tous les passages, c'est-à-dire sa texture `::before` (`hero-bg.jpg`, 3,4 Mo à 4 % d'opacité). Elle n'est découverte qu'au bout de la chaîne d'`@import` et se télécharge en concurrence avec ~10 Mo de photos : en 4G lente, LCP de 66 à 68 s. Métriques locales des DevTools : LCP 2,76 s, CLS 0,02 (voir la figure des métriques locales ci-après).
- **Poids** : « Total size was 17 143 KiB » (Lighthouse DevTools). Économies estimées : 10 977 KiB sur les images, 14 978 KiB sur la mise en cache.
- **Requêtes bloquant le rendu** : 2 680 ms d'économie estimée en mobile, 750 ms en ordinateur.
- **Affichage de la police** (`display=block`) : 350 ms (mobile), 610 ms (ordinateur).
- **Fil principal (mobile)** : 4,6 s de travail, dont 2,9 s d'exécution JavaScript, et 4 tâches longues.
- **Tâches longues (PSI ordinateur, passage 2)** : `ui-kit.min.js` 731 ms à 258 ms, `analytics.js` **464 ms à 10,9 s**, document 89 ms.
- Ajustement forcé de la mise en page, causes de décalage de mise en page, images sans `width`/`height`, erreurs console (404 et preload refusé).

Pourquoi le TBT est bimodal (30 → 510 ms sur la même page). Le TBT ne compte que les tâches situées **entre le FCP et le TTI**. La boucle de 620 ms d'UIKit s'exécute dans le `<head>`, *avant* le premier rendu : elle retarde le FCP et n'entre presque jamais dans le TBT. Ce qui fait basculer le TBT, c'est `analytics.js`. Si son premier passage (~6,5 s + 2,6 s après le `load`) tombe avant le TTI, ce qui arrive quand les images n'ont pas fini de charger, on obtient 360 à 510 ms. Sinon on obtient 30 ms. En mobile (412 px), `analytics.js` ne s'active jamais.

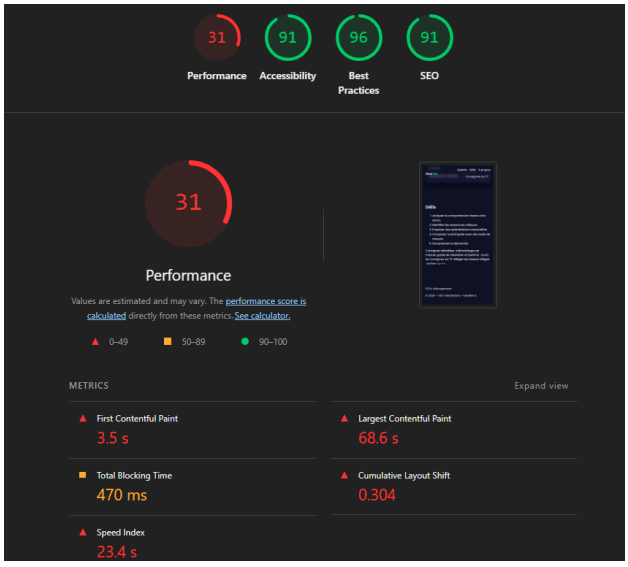


Figure 2 : Lighthouse DevTools, avant : Mobile

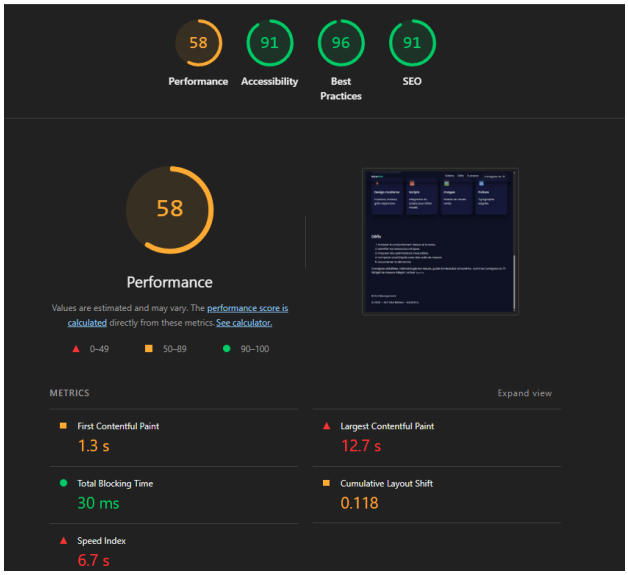


Figure 3 : Lighthouse DevTools, avant : Ordinateur

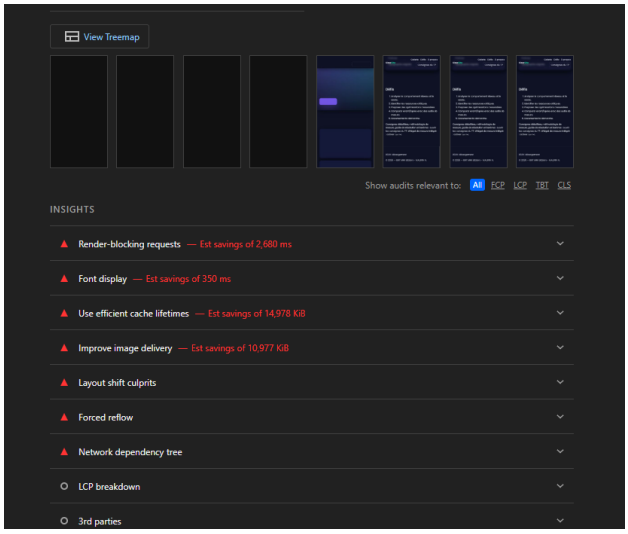


Figure 4 : Insights, avant : Mobile

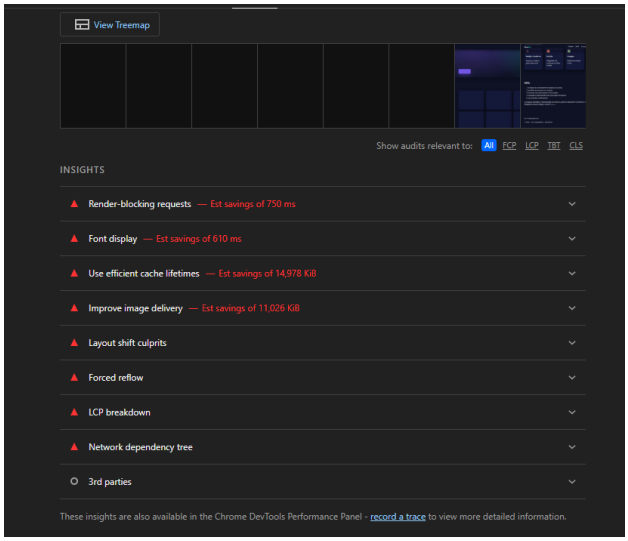


Figure 5 : Insights, avant : Ordinateur

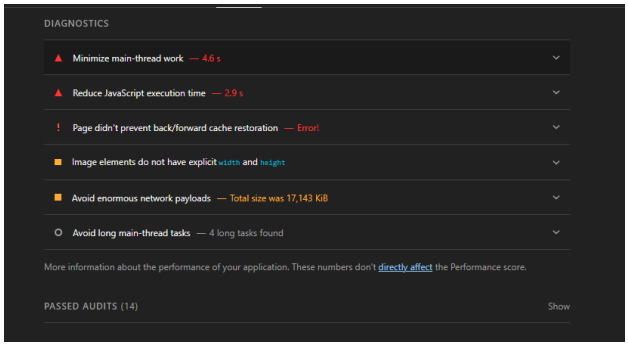


Figure 6 : Diagnostics, avant : Mobile

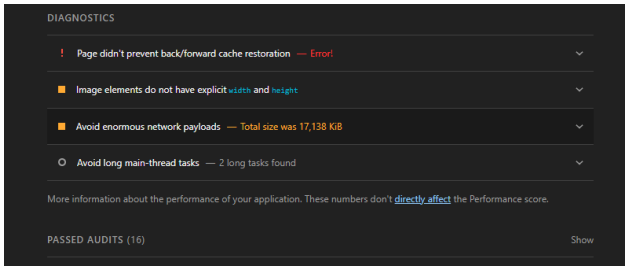


Figure 7 : Diagnostics, avant : Ordinateur

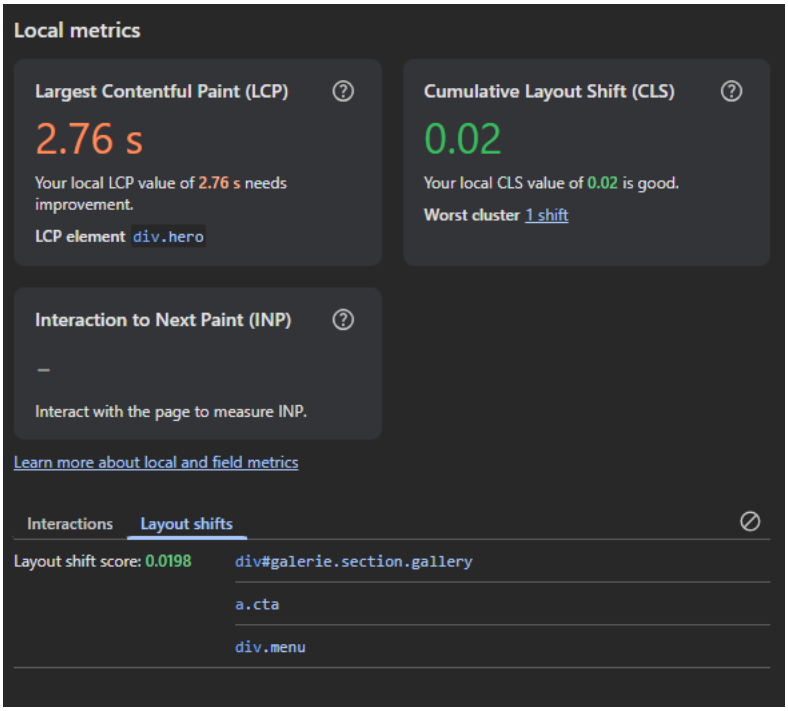


Figure 8 : Métriques locales du panneau Performances, avant

Exécution (avant) : panneau Performances, CPU ×4

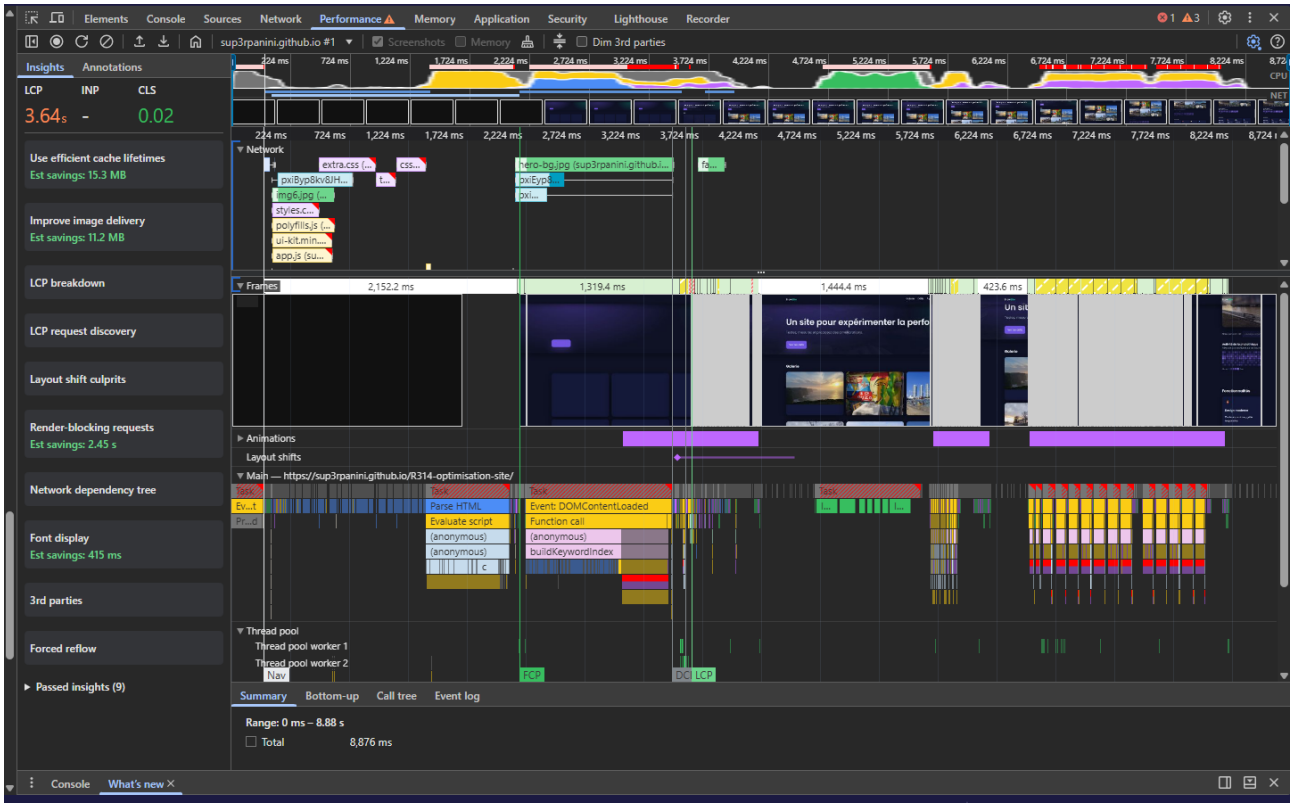


Figure 9 : Panneau Performances, avant : frise (CPU ×4)

La frise montre trois blocs rouges :

- au chargement : *Evaluate script* d'`ui-kit.min.js` ;
- à DOMContentLoaded : `buildKeywordIndex` ;
- pendant le défilement : une tâche à chaque cran (`revealCards`).

Bottom-up, classé par temps propre :

Self time	Total time	Activité	Correspond à
306,8 ms	**610,6 ms (61,4 %)**	`c — ui-kit.min.js:2:26`	P15 (a) : boucle active `_cal`
303,7 ms	303,7 ms	`now`	les appels à `performance.now()` de cette même boucle
49,8 ms	116,1 ms	`set innerHTML`	P16 : `innerHTML +=` des mots-clés
54,1 ms	54,1 ms	Layout	P17/P18 : reflows forcés
15,4 ms	89,4 ms	`getBoundingClientRect`	P17/P18
12,7 ms	22,9 ms	`b — ui-kit.min.js:2:129`	P15 (b) : 240 000 objets `_pool`

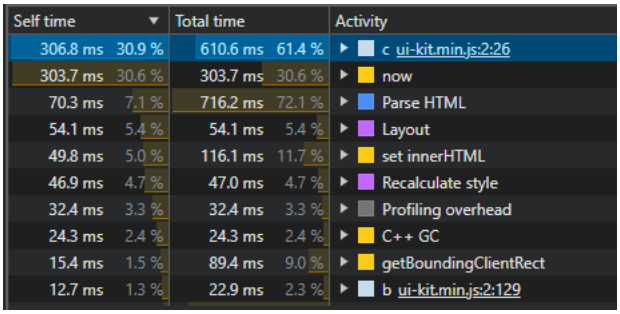


Figure 10 : Panneau Performances, avant : onglet Bottom-up trié par temps propre

Exécution longue (avant) : Chromium 1350×940, sans ralentissement CPU, 30 s

Instant	Blocage	Origine
0,1 s	**687 ms**	`ui-kit.min.js` : boucle active de 620 ms + 240 000 objets
0,7 s	**290 ms**	DOMContentLoaded : index des mots-clés, calendrier, `_sync`
12,7 s	59 ms	`analytics.js` (minuteur)
15,3 s	76 ms	idem
17,9 s	97 ms	idem
20,5 s	124 ms	idem
23,1 s	161 ms	idem
25,7 s	204 ms	idem
28,3 s	262 ms	idem, et la durée continue de croître jusqu'à 430 ms

Chaque événement de défilement coûte en plus 15 à 42 ms, sans aucune limitation de fréquence.

2. Problèmes identifiés

Les numéros de ligne renvoient aux fichiers **d'origine**.

Niveau 1 : réseau et images

P1 — `index.html:75-90`

Nature	Les vignettes affichées à ~388 px chargent des JPEG de 2040 à 3968 px
Preuve	PSI « Améliorer l'affichage des images » (663×374 affiché contre 3968×2240)
Coût	9,9 Mo pour 6 vignettes
Correction	WebP 400/800/1200 + `srcset`/`sizes` calqués sur les media queries

P2 — `index.html:75-90`

Nature	`<img>` sans `width`/`height`
Preuve	PSI « éléments d'image sans width/height »
Coût	Risque de CLS
Correction	`width`/`height` (le ratio réel)

P3 — `index.html:75-90`

Nature	Aucun chargement différé sous la ligne de flottaison
Preuve	Code
Coût	Images 4 à 6 chargées d'emblée
Correction	`loading="lazy"` sur la 2e rangée, `decoding="async"`

P4 — `index.html:12`

Nature	`preload` de `img6.jpg?v=2` : le query string en fait une 2e ressource
Preuve	Réseau : deux lignes img6
Coût	3,4 Mo en double, en priorité haute
Correction	Suppression

P5 — `css/theme-legacy.css:11`

Nature	Texture `hero-bg.jpg` 3968 px / 3,4 Mo à 4 % d'opacité, et **élément LCP**
Preuve	PSI « Détection de la requête LCP » → `div.hero`
Coût	LCP 12,1 s en mobile
Correction	WebP 960 px q40 (45 Ko), rendu indiscernable

P6 — `img/favicon.png`

Nature	Dégradé uni stocké en PNG 512 px bruité
Preuve	Réseau
Coût	787 Ko
Correction	SVG `linearGradient` de 244 octets + repli PNG 32 px (736 o)

P7 — `index.html:16`

Nature	`js/legacy/polyfills.js` n'existe pas, script bloquant
Preuve	Console : 404
Coût	Un aller-retour bloquant
Correction	Suppression

P8 — `index.html:9`

Nature	`dns-prefetch` vers cdn.jsdelivr.net, jamais contacté
Preuve	Réseau : aucun appel
Coût	Résolution DNS inutile
Correction	Suppression

P9 — `css/styles.css:2`

Nature	Google Fonts en `display=block` (texte invisible jusqu'à la police) + graisse 300 jamais utilisée
Preuve	Graisses calculées sur la page : 400 et 700 uniquement
Coût	FOIT, 2 domaines tiers
Correction	Poppins auto-hébergée (400/600), `font-display:swap`

P10 — `index.html:10-11`

Nature	`preload` de police **sans** `crossorigin`, et sur la graisse 300 inutilisée
Preuve	Console : « credentials mode does not match »
Coût	Téléchargement perdu
Correction	Preload des 2 fichiers utilisés, avec `crossorigin`

P11 — `css/styles.css:1`**,** `extra.css:1`**,** `theme-legacy.css:3`

Nature	Chaîne d'`@import` en série (styles → extra → theme-legacy → Google)
Preuve	PSI « Arborescence du réseau »
Coût	4 allers-retours avant le premier rendu
Correction	Une seule feuille

P12 — `styles.css:28-1226`**,** `extra.css:8-806`**,** `theme-legacy.css:18-28`

Nature	1 199 `.redundant-*`, 799 `.extra-*`, `.legacy-*`, `.tile*` et Playfair Display : utilisés dans aucune page
Preuve	Recherche dans tous les HTML et JS
Coût	124 Ko de CSS (10,5 Ko gzip)
Correction	Suppression. Feuille finale : 6 Ko

P13 — `index.html:17-18`

Nature	`ui-kit.min.js` et `app.js` synchrones dans le `<head>`
Preuve	PSI « Requêtes de blocage du rendu »
Coût	Parsing bloqué
Correction	UIKit supprimé (P15), `app.js` en `defer`

P14 — `img1.html` **à** `img6.html`

Nature	L'image s'affiche à sa largeur native : défilement horizontal
Preuve	Page de 2055 px de large sur un écran de 375 px
Coût	Défaut d'affichage + 0,8 à 3,4 Mo
Correction	`max-width:100%` + `srcset` (800/1200/plein format ≤ 2400)

Niveau 2 : profil d'exécution

P15 — `js/vendor/ui-kit.min.js:2`

Nature	Une fois déplié : (a) `_cal` tourne en boucle active 620 ms au parsing ; (b) `_pool` alloue 240 000 objets jamais lus ; (c) un `MutationObserver` sur tout le document (`subtree` + `attributes`) appelle `getComputedStyle` sur **tous** les éléments à chaque mutation
Preuve	Bottom-up : `c` (`ui-kit.min.js:2:26`) 610 ms, soit 61 % du temps. PSI : tâche de 731 à 832 ms
Coût	TBT, mémoire, et chaque écriture de style au scroll relance un recalcul complet sur ~800 nœuds
Correction	`app.js` n'utilisait que `qs`, `qsa`, `cls`, `ready` : remplacés par le DOM natif, UIKit supprimé

P16 — `js/app.js:81`

Nature	`list.innerHTML +=` dans une boucle de 320 mots-clés : la liste est reparsée à chaque tour
Preuve	Code + tâche de 290 ms au DOMContentLoaded
Coût	Quadratique
Correction	`DocumentFragment`, une seule insertion

P17 — `js/app.js:161-172`

Nature	Pour chaque carte : `querySelectorAll('.card')` + lecture de `offsetHeight` de toutes les cartes juste après l'écriture d'un `min-height`
Preuve	PSI « Ajustement forcé », `app.js:162/174`
Coût	n^2 + reflow forcé, à chaque scroll
Correction	Calcul seulement au chargement et au redimensionnement : lectures, puis écritures

P18 — `js/app.js:183-188` **+** `:213-215`

Nature	Pour les 371 cases du calendrier : `getBoundingClientRect()` puis `style.opacity` en alternance, à chaque événement scroll non limité
Preuve	PSI : 163 ms sur `app.js:185`
Coût	371 reflows forcés par événement, amplifiés par P15 (c)
Correction	Les 7 rangées sont mesurées une fois, une seule lecture par image, `requestAnimationFrame`, écouteur `passive`

Niveau 3 : conditionnel et différé

P19 — `js/analytics.js:19-29`

Nature	Code obscurci (`atob` → `innerWidth`, `performance`, `now`, `hidden`). Il démarre 6,5 s après le `load`, puis s'exécute toutes les 2,6 s, **seulement si la fenêtre fait plus de 1024 px et que l'onglet est visible**. Il sérialise tout le DOM puis tourne en boucle active : 46 ms ×1,28 à chaque passage, jusqu'à 430 ms
Preuve	PSI ordinateur : tâche de 464 ms à 10,9 s attribuée à `analytics.js`. Mesure locale à 1350 px : 59 → 262 ms en 30 s, rien à 1024 px. Explique le TBT bimodal (30 / 360 / 510 ms) et l'écart Mobile/Ordinateur
Coût	Le site rame après quelques secondes sur ordinateur portable : c'est la plainte du client
Correction	Suppression. Il n'envoie rien (aucune requête, résultat rangé dans `window.__au`, lu nulle part) : aucune fonctionnalité perdue

3. Résultats (après)

Mêmes outils et mêmes réglages que pour l'« avant ».

Les six indicateurs

Indicateur	Avant	Après	Évolution	Source
Poids transféré	17,6 Mo	**454 kB**	−97 %	DevTools Réseau
Requêtes	23	**16**	−7	DevTools Réseau
FCP (mobile)	1,7 s	**0,8 s**	−53 %	PSI
LCP (mobile)	66,8 s	**1,8 s**	−97 %	PSI
CLS (mobile)	0,011	**0**		PSI
TBT (mobile)	220 ms	**0 ms**	−100 %	PSI (ordinateur : 510 → 0 ms)

PageSpeed Insights (Lighthouse 13.6.0, avant à 21 h 56, après à 22 h 12)

Profil		Score	FCP	LCP	TBT	CLS	Speed Index
Mobile	avant	60	1,7 s	66,8 s	220 ms	0,011	17,5 s
Mobile	**après**	**98**	**0,8 s**	**1,8 s**	**0 ms**	**0**	**3,8 s**
Ordinateur	avant	43	1,0 s	12,2 s	510 ms	0	4,2 s
Ordinateur	**après**	**100**	**0,2 s**	**0,3 s**	**0 ms**	**0**	**0,4 s**

Rapport PSI après : mobile, ordinateur.

Lighthouse dans les DevTools (13.4.1)

Profil		Score	FCP	LCP	TBT	CLS	Speed Index
Mobile	avant	31	3,5 s	68,6 s	470 ms	0,304	23,4 s
Mobile	**après**	**86**	**1,0 s**	**2,1 s**	530 ms	**0,011**	**1,2 s**
Ordinateur	avant	58	1,3 s	12,7 s	30 ms	0,118	6,7 s
Ordinateur	**après**	**100**	**0,3 s**	**0,5 s**	**10 ms**	**0**	**0,4 s**

Les audits encore signalés après correction : « durées de mise en cache » (en-têtes GitHub, voir §4), « découverte de la requête LCP » (texture découverte via la CSS, voir §4) et quelques Kio d'images estimés récupérables.

Le TBT mobile des DevTools (530 ms) ne vient pas du site. PSI, sur les serveurs de Google, mesure **0 ms** pour la même version. En local, la frise Performances attribue 1 467 ms de fil principal à « [unattributed] » et fait apparaître les extensions du navigateur (Vivaldi, Picture-in-Picture), alors que le script du site ne totalise que 172 ms sur tout l'enregistrement. Le mode mobile des DevTools multiplie ces durées locales par le ralentissement simulé. C'est l'écart que la méthodologie annonçait entre une machine personnelle et PSI, et c'est pourquoi PSI sert de référence.

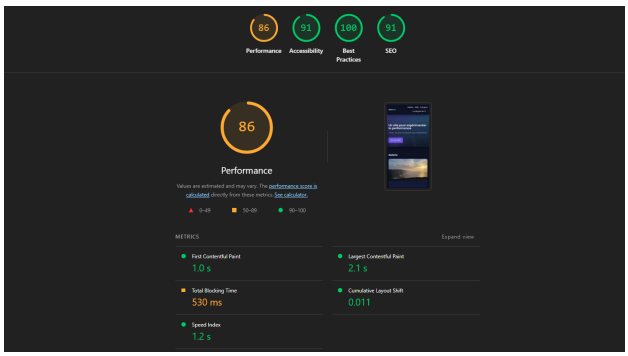


Figure 11 : Lighthouse DevTools, après : Mobile

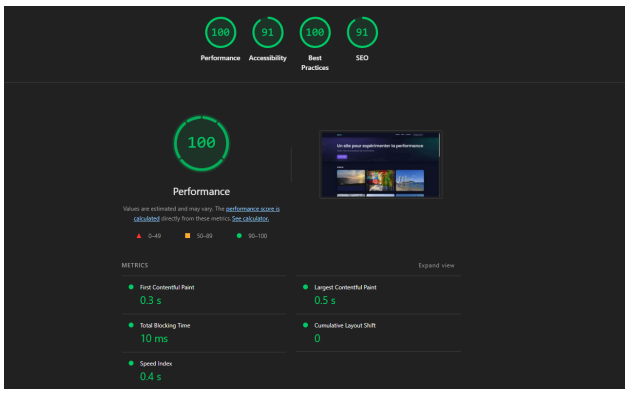


Figure 12 : Lighthouse DevTools, après : Ordinateur

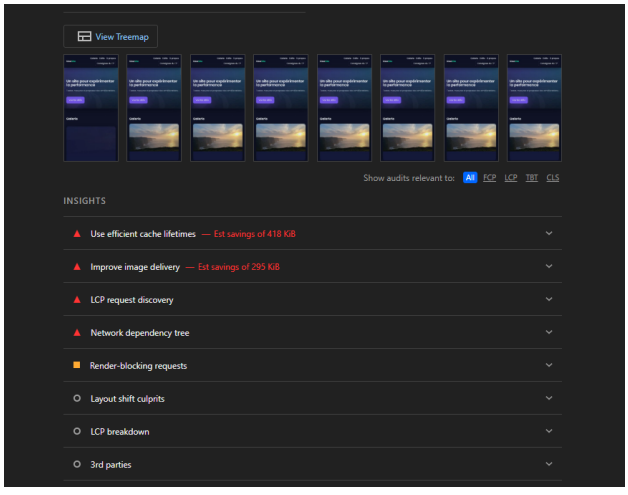


Figure 13 : Insights, après : Mobile

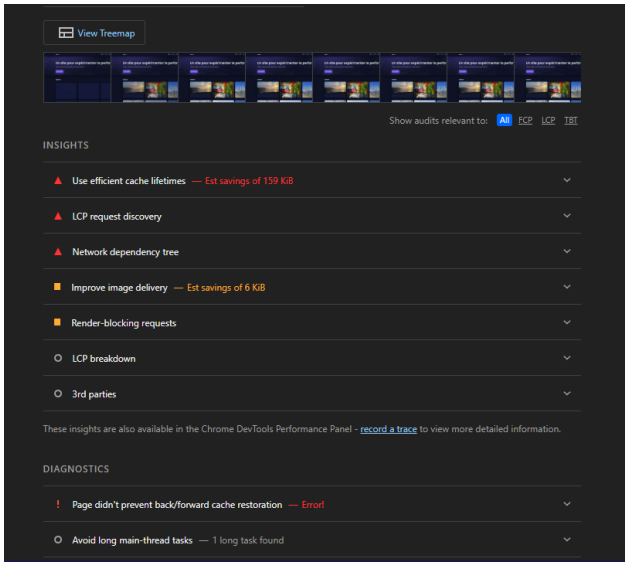


Figure 14 : Insights et diagnostics, après : Ordinateur

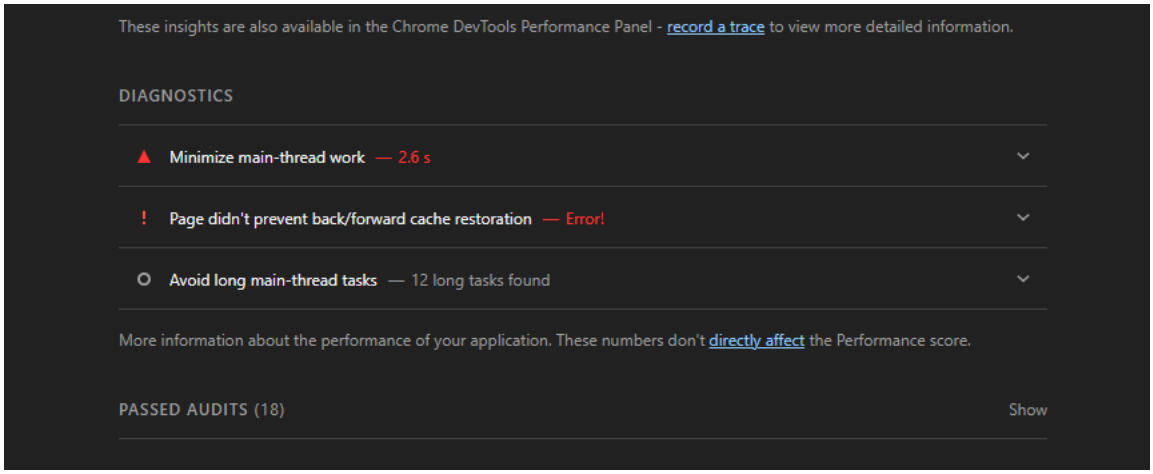


Figure 15 : Diagnostics, après : Mobile

DevTools Réseau et Performances

Mesure	Avant	Après
Requêtes	23	16
Transféré / ressources	17,6 Mo / 17,7 Mo	**454 kB / 470 kB**
DOMContentLoaded	3,47 s	**502 ms**
Load	3,58 s	**574 ms**
LCP (panneau Performances)	3,64 s	**0,65 s**
CLS (panneau Performances)	0,02	**0**

Sur la frise après, la tâche d'`ui-kit.min.js` et le bloc `buildKeywordIndex` du chargement ont disparu.

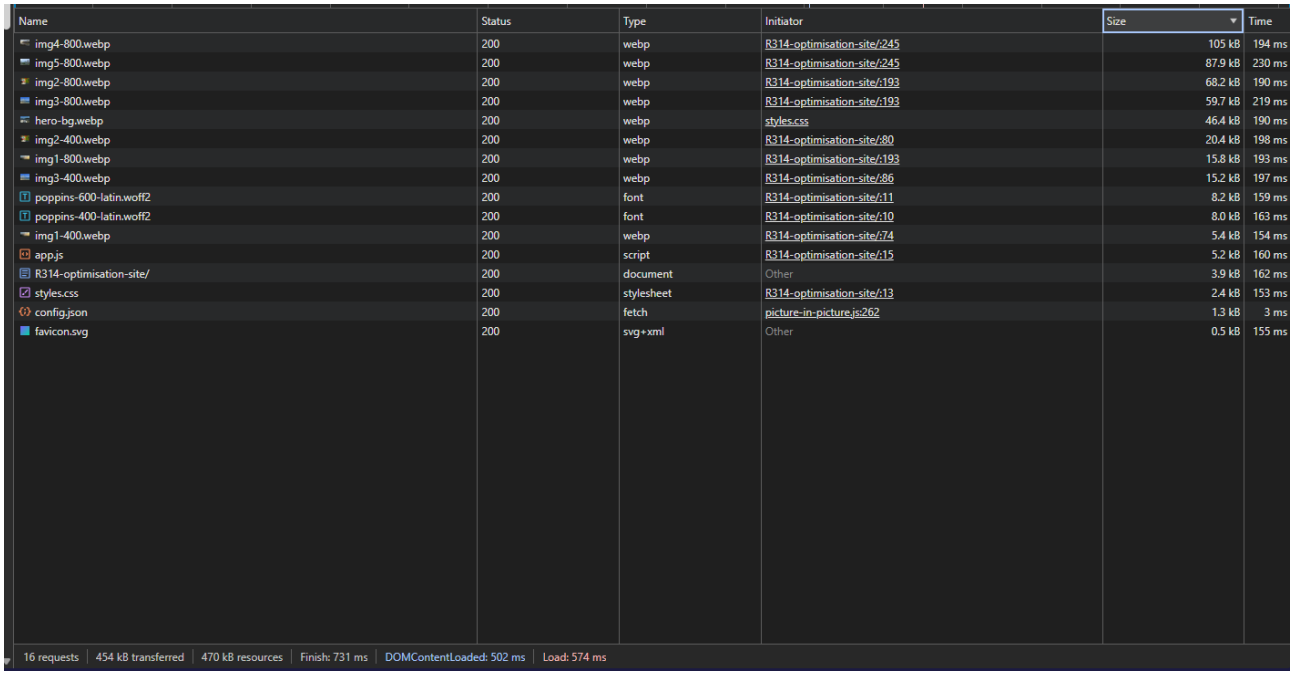


Figure 16 : Onglet Réseau, après : tri par taille décroissante

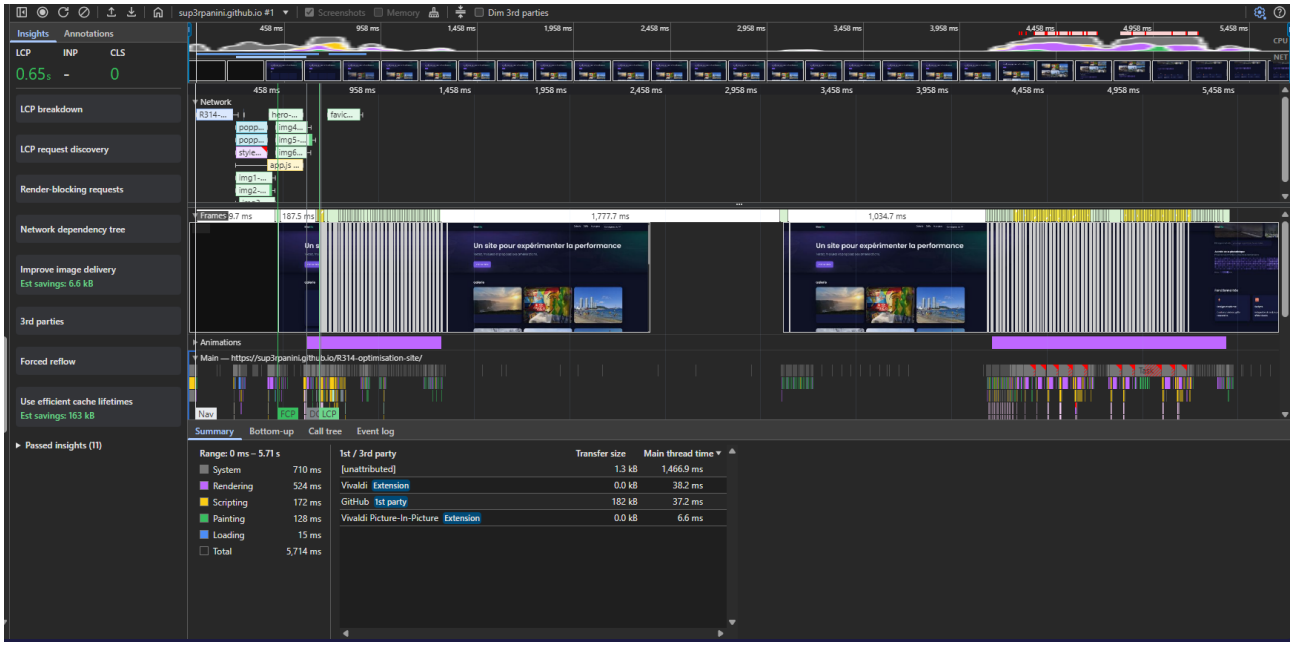


Figure 17 : Panneau Performances, après : frise (CPU ×4)

Ce qui reste. Des images un peu longues subsistent pendant le défilement. Ce n'est plus du script (172 ms de *Scripting* au total) mais du rendu (524 ms de *Rendering*). Le calendrier anime l'opacité de ses 371 cases à chaque image, avec une transition CSS. C'est un effet visuel du site, conservé pour l'iso-rendu. Piste non appliquée : isoler le calendrier (`contain: paint`) pour limiter la zone repeinte.

Mesures complémentaires

Indicateur	Avant	Après	Mesure
Tâches > 50 ms (30 s, scroll, 1350 px)	9 (de 59 à 687 ms)	**0** (max 12 ms)	Chromium local, même protocole
CSS	124 Ko en 3 fichiers + 2 CSS tierces	6 Ko, 1 fichier	
JS	3 fichiers + 1 404	1 fichier en `defer`	
CLS au pire cas (polices +1,5 s, images +1 s)	—	0 en 1350 px et en 375 px	Serveur local volontairement ralenti

Vérification de l'iso-rendu

- Fenêtre 1350×940 : position et taille de **tous** les éléments de la page (plus de 100), ombres, graisses et opacités **identiques** entre l'original et la version corrigée.
- Opacités des 371 cases du calendrier après défilement : identiques à 6 décimales près.
- Mobile 375 px : filtre (« heure dorée » → « 1 visuel sur 6 », recherche sans résultat), ancre « Voir les défis », état des cartes et `min-height` : identiques.
- **Seul changement visible voulu** : sur les pages `imgN.html`, l'image est désormais bornée à la largeur de la page au lieu de déborder (défaut A1 du guide).

4. Hors de portée sur GitHub Pages (avec un vrai serveur)

- **Cache** : GitHub impose `Cache-Control: max-age=600`, ce qui explique l'audit PSI « durées de mise en cache » toujours en échec. Je mettrais `max-age=31536000, immutable` sur les images, polices, CSS et JS versionnés par une empreinte dans le nom (`styles.a3f9c1.css`, ce qu'un outil de build fait proprement), et une revalidation courte sur le HTML.
- **Compression** : Brotli plutôt que gzip pour le HTML, le CSS et le JS (environ 15 à 20 % de mieux). Inutile sur WebP et woff2.
- **Indice de chargement** : en-tête `Link: rel=preload` pour la feuille de style, ou `103 Early Hints`.
- **Ce qui n'a pas été fait volontairement** : précharger `hero-bg.webp` améliorerait le LCP mesuré, mais cela donnerait la priorité à une texture quasi invisible. Le `backdrop-filter` du bandeau est conservé (iso-rendu).